ATS App Summary

One-page repo-backed overview generated from the codebase on 2026-03-26

What It Is

AASTHIX TALENT is a Next.js-based applicant tracking workspace for managing jobs, candidates, applications, interviews, and recruiting operations.

The repo also includes optional AI-assisted matching and screening flows plus a separate Python matcher service for deterministic JD-resume scoring.

Who It's For

Primary persona: internal recruiting teams - especially recruiters and admins managing requisitions, candidate flow, interviews, and team coordination.

What It Does

- Manage jobs, requisitions, candidates, vendors, and applications.
- Track pipeline stages, interview schedules, and interview alerts.
- Show dashboard, funnel, recruiter, and screening analytics.
- Support role-based access, admin permissions, invites, and audit views.
- Provide team chat, activity center, and recruiter copilot/workload screens.
- Publish a careers portal and accept job applications and resume parsing imports.
- Run candidate-job matching through rule-based, queued, and optional OpenAI/vector flows.

How To Run

- 1. Run `npm install`.
- 2. Create `.env.local` from `.env.example`; set `DATABASE_URL`, `JWT_SECRET`, and other needed values.
- 3. Provision PostgreSQL and apply SQL files in `migrations/` manually. Migration runner command: Not found in repo.
- 4. Run `npm run dev`, then open `http://localhost:3000`.
- 5. Optional: start `matcher-service` for `/api/match/no-ai`; start Redis plus `npm run worker:ai-match` for async AI matching.

How It Works

- UI: Next.js App Router pages under `app/(dashboard)`, `app/(auth)`, and `app/careers`; shared React components under `components/` call JSON APIs with `apiFetchJson` and SWR providers.
- Auth + access: middleware checks for the auth cookie on protected pages; API routes validate JWT-backed users and enforce role/permission gates through `lib/auth*` and `lib/rbac.ts`.
- Core backend: route handlers in `app/api/**` read and write ATS data with `node-postgres` via `lib/db.js`, backed by PostgreSQL schema files in `migrations/`.
- Async matching: BullMQ queues on Redis enqueue job-level matching runs; `lib/queue/worker.ts` executes `runSkillProfileExtractCore` and persists run status.
- External/optional services: `matcher-service/` exposes a Python `/match` endpoint for deterministic JD-resume scoring; OpenAI/vector features are controlled by env flags in `.env.example` and matching libs under `lib/`.
- Infra/deployment docs: Not found in repo.

Sources used: package.json, README.md, .env.example, app/, lib/, matcher-service/, migrations/